

RA01_UD02 Tarea 1: desarrollo de interfaces básicas



Índice

Explicación:	3
Capturas:	7
Enlace repositorio Github con zip del proyecto:	14
Ejecución del proyecto:	14

Explicación:

Hecho con React con Typescript adornado con CSS normal para el tema de las ventanas y para las interfaces HTML y CSS normal.

Tipos de contenedores / Layouts:

Home.html

Contenedores utilizados:

`<header>`: layout superior que agrupa el logotipo, la navegación principal y las acciones (selector de idioma y botones).

Motivo: delimita la cabecera común del sitio, mejora la semántica y accesibilidad.

`<nav>`: contenedor interno dentro del header para los enlaces principales.

Motivo: define claramente la navegación principal para lectores de pantalla.

`<main>`: contenedor central del contenido principal (la sección hero).

Motivo: separa el contenido principal de la cabecera y el pie de página.

`<section class="hero-section">`: bloque visual de llamada a la acción.

Motivo: agrupa título, descripción y botones dentro de una misma unidad de diseño.

`<div>`: se emplea para subdividir elementos dentro del header (logo, idioma, acciones) y para el bloque .hero-actions.

Motivo: facilita la organización flexible con Flexbox o Grid, y permite aplicar estilos específicos.

Resumen del layout:

Estructura en rejilla vertical con un header fijo y un main centralizado (hero). Ideal para landing pages.

Login.html

Contenedores utilizados:

`<main>`: estructura base de la interfaz de autenticación.

Motivo: identifica el contenido principal y mejora la semántica.

`<form>`: contenedor de los campos de entrada del usuario.

Motivo: agrupa y gestiona la información del formulario, permite validación.

`<div>`: empleado en grupos (`.form-group`, `.login-header`, `.group`) para estructurar los campos y aplicar estilos.

Motivo: permite distribuir los elementos mediante layout vertical centrado.

`<p>` y `<a>`: contenedores menores para texto auxiliar y enlaces secundarios.

Resumen del layout:

Estructura centrada en columna, con un contenedor principal que agrupa el formulario y elementos decorativos. Uso intensivo de contenedores `<div>` con clases específicas para un diseño modular.

Signup.html

Contenedores utilizados:

`<main>`: contenedor global del contenido principal.

Motivo: semántica HTML5 y accesibilidad.

`<form>`: estructura base para la creación de cuenta.

Motivo: agrupa campos relacionados con registro, facilita validación.

`<div class="form-group">`: layout modular para cada campo (etiqueta + input).

Motivo: alineación coherente de etiquetas y campos, facilita estilos adaptables.

`<div class="password-wrapper">`: contenedor específico que combina input y un icono.

Motivo: diseño funcional (mostrar/ocultar contraseña o reforzar la identidad visual).

`<p>` y `<a>`: texto informativo y enlaces de redirección.

Resumen del layout:

Diseño en bloque vertical con jerarquía clara (logo → formulario → acción secundaria). Uso de contenedores anidados para estilos granulares.

Dashboard.html

Contenedores utilizados:

`<div class="dashboard">`: layout principal que actúa como contenedor raíz.

Motivo: permite aplicar un layout de dos columnas (sidebar + contenido).

`<aside>`: barra lateral izquierda.

Motivo: semántica para navegación secundaria, uso persistente en la app.

`<main>`: contenedor del área de trabajo o contenido dinámico.

Motivo: define el área principal de interacción.

`<header class="page-header">`: encabezado de página dentro del main.

Motivo: separa título, subtítulo y acciones globales.

`<section>` y `<article>`: subdividen estadísticas y bloques funcionales.

Motivo: estructura semántica para los diferentes tipos de datos o componentes visuales.

`<div>`: agrupaciones para iconos, textos o botones dentro de cada card o sección.

Motivo: soporte para layouts de rejilla o flexibles en cada subsección.

Resumen del layout:

Estructura de aplicación administrativa, con sidebar fija y área de contenido fluida. Combina contenedores semánticos y estructurales para mantener jerarquía y accesibilidad.

Tipos de controles de interfaz y justificación:

Tipo de Control	Dónde se usa	Función / Justificación
Botones (<code><button></code>)	En todas las interfaces (home, login, signup, dashboard)	Ejecutan acciones directas: “Probar demo”, “Entrar”, “Crear cuenta”, “Añadir empleado”, etc. Su uso mejora la interacción táctil y la accesibilidad.
Campos de texto (<code><input type="email"></code>, <code><input type="password"></code>)	Login y Signup	Recoger credenciales del usuario. Los tipos email y password activan validaciones y teclado adaptado en móviles.
Enlaces (<code><a></code>)	En navegación principal (home, dashboard) y enlaces auxiliares (login/signup)	Permiten la navegación entre secciones o vistas relacionadas. Su semántica favorece la accesibilidad.
Menú de navegación (<code><nav></code>)	Home y Dashboard	Estructura los enlaces principales del sitio o aplicación. Mejora la organización y usabilidad.
Selectores de idioma (ícono + texto)	Home (header)	Simula un control de tipo desplegable para cambio de idioma. Mejora la internacionalización y UX global.
Iconos / Imágenes (<code></code>, <code><svg></code>)	En todos los HTML	Refuerzan la identidad visual y mejoran el reconocimiento rápido de acciones (notificaciones, menú, etc.).

Etiquetas <label> asociadas a inputs	Login y Signup	Vinculan texto descriptivo con los campos, esenciales para accesibilidad y compatibilidad con lectores de pantalla.
Secciones y artículos (<section>, <article>)	Home y Dashboard	Separan bloques temáticos (hero, estadísticas, gráficos, acciones rápidas). Facilitan la lectura y el mantenimiento del layout.

Capturas:

Diseño de Home

Abrir ventana

Diseño de Login

Abrir ventana

Diseño de Signup

Abrir ventana

Diseño de Dashboard

Abrir ventana

Diseño de Home

Diseño de Login

Diseño de Signup

Diseño de Dashboard



Diseño de Home

Diseño de Login

Diseño de Signup

Diseño de Dashboard

Login



jhandler

TalentoCorp S.L.

Correo electrónico

usuario@talentocorp.com

Contraseña

Entrar

[¿Olvidaste tu contraseña?](#)

[¿No tienes una cuenta? Registrarse](#)

Cerrar ventana

Diseño de Home

Diseño de Login

Diseño de Signup

Diseño de Dashboard

Signup



jhandler

TalentoCorp S.L.

Correo electrónico

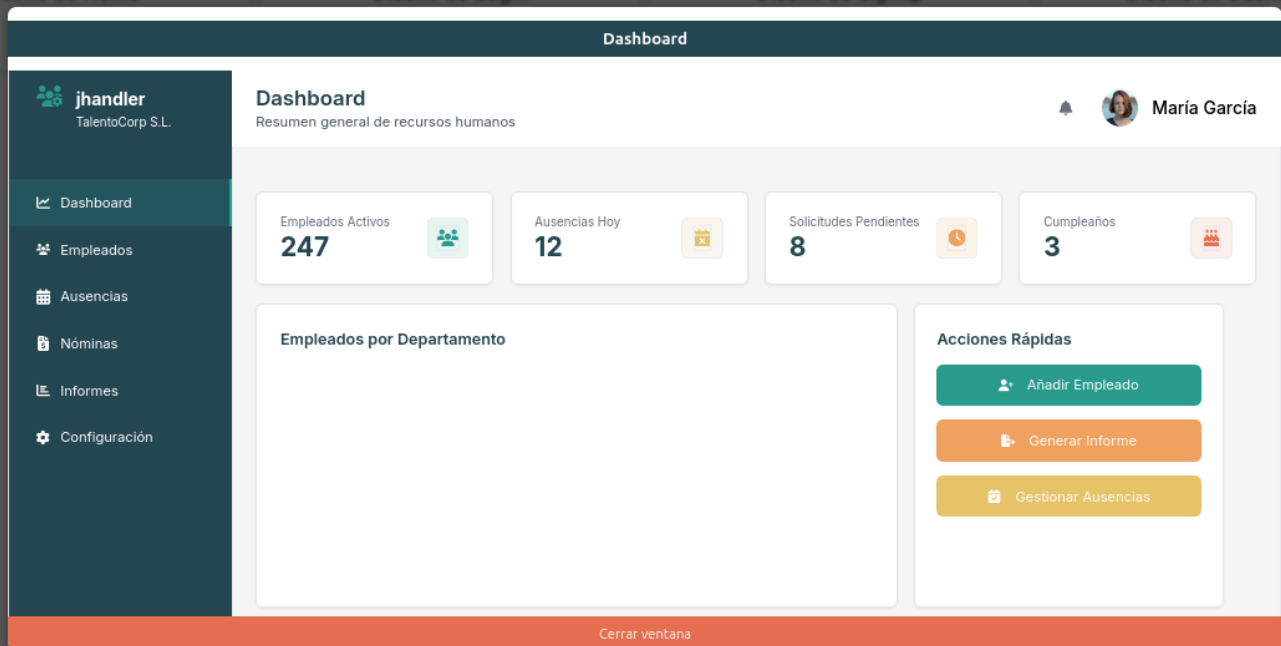
usuario@talentocorp.com

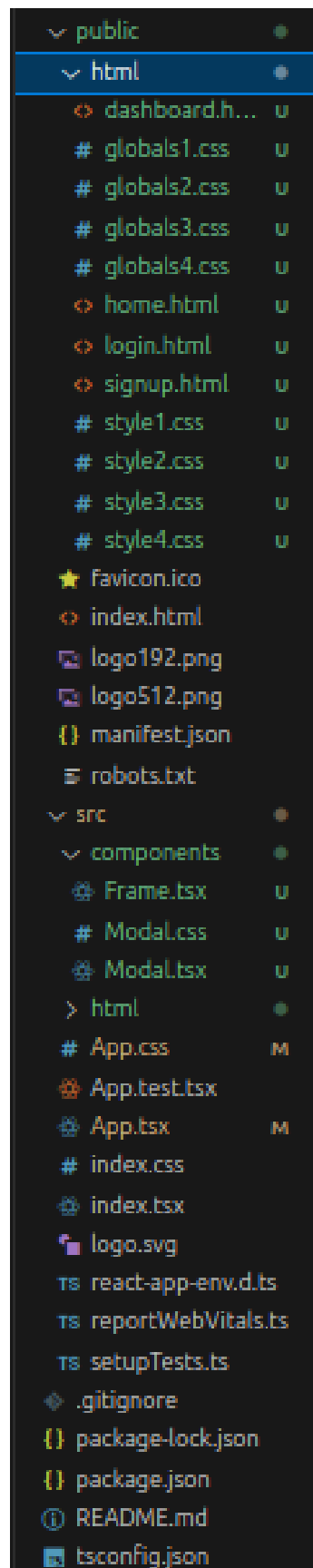
Contraseña

Crear cuenta

[¿Ya tienes cuenta? Iniciar sesión](#)

Cerrar ventana





```

Modal.tsx u x
src > components > Modal.tsx > ...
1  import React, { useState, useEffect } from "react";
2  import "../Modal.css";
3
4  interface ModalProps {
5    show: boolean;
6    onClose: () => void;
7    htmlPath: string;
8    title: string;
9  }
10
11  const Modal: React.FC<ModalProps> = ({ show, onClose, htmlPath, title }) => {
12    const [closing, setClosing] = useState(false);
13
14    useEffect(() => {
15      if (!show) setClosing(false);
16    }, [show]);
17
18    const handleClose = () => {
19      setClosing(true);
20      setTimeout(() => {
21        setClosing(false);
22        onClose();
23      }, 250); // coincide con duración del fadeOut
24    };
25
26    if (!show && !closing) return null;
27
28    return (
29      <div className={`modal-overlay ${closing ? "fadeOut" : ""}`}>
30        <div className={`modal-content ${closing ? "zoomOut" : ""}`>
31          <h2 className="modal-title">{title}</h2>
32          <iframe
33            src={htmlPath}
34            title={title}
35            className="modal-iframe"
36            sandbox="allow-same-origin allow-scripts allow-popups allow-forms"
37          ></iframe>
38          <button className="close-btn" onClick={handleClose}>
39            Cerrar ventana
40          </button>
41        </div>
42      </div>
43    );
44  };
45
46  export default Modal;
47

```

```
Frame.tsx u x
src > components > Frame.tsx > ...
1  import React, { useState } from "react";
2  import Modal from "../Modal";
3
4
5  interface FrameProps {
6    title: string;
7    htmlFile: string;
8  }
9
10 const Frame: React.FC<FrameProps> = ({ title, htmlFile }) => {
11   const [showModal, setShowModal] = useState(false);
12
13   return (
14     <div className="frame">
15       <h2>Diseño de {title}</h2>
16       <button className="open-btn" onClick={() => setShowModal(true)}>
17         Abrir ventana
18       </button>
19       <Modal
20         show={showModal}
21         onClose={() => setShowModal(false)}
22         htmlPath={htmlFile}
23         title={title}
24       />
25     </div>
26   );
27 };
28
29 export default Frame;
30
```

```

App.tsx 1 X
src > App.tsx > ...
1 import React from "react";
2 import Frame from "../components/Frame";
3 import "../App.css";
4
5 function App() {
6   const base = process.env.PUBLIC_URL || "";
7
8   return (
9     <div className="app-container">
10       <Frame title="Home" htmlFile={`${base}/html/home.html`} />
11       <Frame title="Login" htmlFile={`${base}/html/login.html`} />
12       <Frame title="Signup" htmlFile={`${base}/html/signup.html`} />
13       <Frame title="Dashboard" htmlFile={`${base}/html/dashboard.html`} />
14     </div>
15   );
16 }
17
18 export default App;
19

```

```

index.tsx X
src > index.tsx > ...
1 import React from 'react';
2 import ReactDOM from 'react-dom/client';
3 import '../index.css';
4 import App from './App';
5 import reportWebVitals from './reportWebVitals';
6
7 const root = ReactDOM.createRoot(
8   document.getElementById('root') as HTMLElement
9 );
10 root.render(
11   <React.StrictMode>
12     <App />
13   </React.StrictMode>
14 );
15
16 // If you want to start measuring performance in your app, pass a function
17 // to log results (for example: reportWebVitals(console.log))
18 // or send to an analytics endpoint. Learn more: https://bit.ly/CRA-vitals
19 reportWebVitals();
20

```

Enlace repositorio Github con zip del proyecto:

<https://github.com/RafaelMayor/DAD/tree/main/actividad-ventanas-jhandler>

Ejecución del proyecto:

AL DESCOMPRIMIR EL CÓDIGO PARA PODER
EJECUTARLO PRIMERO EJECUTAR EL COMANDO
DENTRO DEL PROYECTO:

`npm install`

Y una vez ya esté la carpeta node_modules EJECUTAR EL
COMANDO DENTRO DEL PROYECTO:

`npm start`